

Konputazio Ingeniaritza eta
Sistema Adimentsuak
Unibertsitate Masterra

Máster Universitario en
Ingeniería Computacional
y Sistemas Inteligentes

INGENIERÍA COMPUTACIONAL Y SISTEMAS INTELIGENTES

Evitador de obstáculos

Robótica Inteligente

Estudiantes: Eneko Perez, Alan García y Alex Amenabar

Curso: 2024-2025

Fecha: 9 de marzo de 2025

Índice

1. Análisis del problema	2
2. Estrategia utilizada y justificación de la misma	2
3. Algoritmo de la implementación	3
4. Evaluación del comportamiento	4
5. Mejoras posibles	6

1. Análisis del problema

En este proyecto se aborda el problema de implementar un módulo de comportamiento para un robot móvil que le permita desplazarse de manera autónoma evitando obstáculos en su entorno. Para ello, el robot debe utilizar los datos proporcionados por su sensor LiDAR, los cuales permiten detectar la presencia y distancia de objetos alrededor del robot, en todas las direcciones.

El desafío principal radica en definir una estrategia eficiente que permita al robot moverse de forma fluida por el entorno sin colisionar con los obstáculos, considerando las limitaciones del sensor. Además, es necesario garantizar que el robot sea capaz de reaccionar a diferentes situaciones, como pasillos estrechos, esquinas, o objetos que se mueven, de manera inteligente y estable.

2. Estrategia utilizada y justificación de la misma

La estrategia utilizada para la tarea se basa en las lecturas del LiDAR, un láser que indica la distancia hacia los objetos del entorno. En este caso, el LiDAR proporciona lecturas de todo el entorno alrededor del robot.

La estrategia se basa en la detección de la mejor ventana de avance utilizando los datos del sensor LiDAR del robot. La ventana se define como la región con mayor espacio libre dentro del campo de visión frontal del robot, lo que le permite moverse de manera segura sin colisionar con obstáculos.

Para lograr esto, el robot analiza continuamente los datos proporcionados por el sensor LiDAR en un rango de 180° al frente. A partir de esta información, identifica las áreas despejadas y aquellas ocupadas por obstáculos. Con estos datos, se identifican las ventanas válidas y se selecciona la mejor entre ellas. Las ventanas válidas son aquellas secuencias contiguas de las lecturas LiDAR donde las distancias superan el umbral. La mejor entre todas, es la ventana donde la suma de las distancias tiene mayor valor. De esta manera, la ventana hacia la que avanza el robot busca un balance entre la lejanía de los obstáculos y la amplitud del área libre. Algunos ejemplos de la búsqueda de ventanas se reflejan en las figuras 1 y 2.

Una vez identificada la mejor ventana o área a la que avanzar, la dirección exacta en la que avanza el robot es la que corresponde al punto central de la ventana. Además, también ajusta la velocidad en función de la distancia media calculada a partir de la secuencia de distancias de la ventana, yendo así más lento cuanto más cerca está de los obstáculos, y más rápido cuanto más lejos esté. Para esto, se normaliza la distancia media en un valor entre 0 y 1 dividiendo la media por una distancia máxima establecida (ver el algoritmo 2). Ese valor es el que establece la velocidad lineal del robot.

La decisión de utilizar solamente las lecturas frontales tiene como razón obtener un comportamiento más fluido e inteligente en los movimientos del robot, ya que en el caso contrario la búsqueda de la mejor ventana puede dar como resultados ventanas muy alejadas entre sí en un espacio de tiempo muy corto, generando un comportamiento brusco.

Para terminar con el manejo de la velocidad lineal, en caso de que el robot tenga muchos obstáculos cerca y no sea capaz de encontrar ninguna ventana válida, se establece el valor 0 en la velocidad lineal, y el robot empieza a girar, lo cual nos lleva a la gestión de la velocidad angular.

3 ALGORITMO DE LA IMPLEMENTACIÓN

Al avanzar hacia la mejor ventana o área encontrada, el robot ajusta su trayectoria girando hacia el punto central de la ventana. Sin embargo, si no identifica ninguna ventana viable dentro de su campo de visión frontal y a una distancia de un metro (nuestro umbral de seguridad), se detiene, retrocede y reorienta su posición para buscar una nueva ruta. Este mecanismo le permite navegar de manera eficiente en entornos con obstáculos impredecibles o pasillos estrechos.

Esta estrategia permite un comportamiento reactivo eficiente basado en información sensorial en tiempo real. De esta manera, el robot es capaz de evitar colisiones y adaptarse a entornos dinámicos sin la necesidad de una planificación global compleja.

3. Algoritmo de la implementación

El código que implementa lo mencionado en el apartado anterior se encuentra en un repositorio de GitHub, concretamente en este enlace. La implementación de la lógica del robot consiste en dos funciones: *detect_obstacle()* y *find_largest_window()*. Las características más importantes de estas se pueden apreciar en los algoritmos 1 y 2.

Algorithm 1 Algoritmo general para detectar los obstáculos y decidir qué hacer en cada situación.

```

fov_indices = índices de los valores del scan frontales del robot
danger_indices = índices del frente del robot
fov_scan = scan[fov_indices]
fov_bearings = bearings[fov_indices]
danger_scan = scan[danger_indices]
start, end, value = find_largest_window(fov_scan, threshold, max_distance)
target_index = start + (end - start) // 2
target_angle = fov_bearings[target_index]
danger_value = min(danger_scan)
if danger_value < threshold then
    speed = 0.1
    turn = target_angle * 2
    if No encuentra ninguna mejor opción then
        speed = -0.25
    end if
else
    speed = value
    turn = target_angle
end if

```

En el algoritmo 1, las primeras líneas consisten en obtener los índices de los valores escaneados que nos interesan. Las listas fov son listas filtradas donde se guardan los valores de distancias y rotaciones que corresponden a la parte frontal del robot. A su vez, la lista danger_scan guarda los valores de distancias de una pequeña parte frontal del robot. Esto se utiliza para evitar que siga avanzando en caso de tener un obstáculo muy cerca. La función *find_largest_window* es la función que encuentra la mejor ventana, y devuelve los índices de inicio y fin de la ventana, y la media de las distancias de la misma. Luego, en target_index se guarda el índice del punto central de la ventana. Utilizando esa información se decide el comportamiento del robot. En la condición, si el robot está dentro de la denominada zona de peligro, para y empieza a girar. En caso de que el target_angle no esté permitiendo que el robot gire, retrocede. Por último, en caso de que el robot no esté en la zona peligrosa, simplemente avanza hacia el punto central de la ventana y va girando para acercarse a esta.

4 EVALUACIÓN DEL COMPORTAMIENTO

Algorithm 2 Algoritmo de la función para encontrar la mejor ventana.

Require: sequece: list, trehsold: float, max_distance: float

```

current_sum = 0
max_sum = 0
start = 0
end = 0
temp_start = 0
for i en len(sequence) do
  if sequence[i] > trehsold then
    current_sum += min(sequence[i], max_distance)
  else
    if current_sum > max_sum then
      max_sum = current_sum
      start = temp_start
      end = i-1
    end if
    current_sum = 0
    temp_start = 1
  end if
end for
if end - start > 0 then
  value = (max_sum / (end-start)) / max_distance
else
  value = 0
end if
return(start_index: int, end_index: int, value: float)

```

La función para encontrar la mejor ventana es la que se refleja en el algoritmo 2. En esta función, a lo largo del bucle se van buscando ventanas. Cuando un valor no supera el umbral, la ventana se termina y se calcula la media. Si la suma de distancias de esa ventana es mayor que la suma de la mejor ventana encontrada hasta el momento, se sustituye. Es posible que la función no encuentre ninguna ventana.

4. Evaluación del comportamiento

Para analizar el comportamiento del robot, se presentan las siguientes figuras que ilustran las lecturas del sensor LiDAR en distintos escenarios. En el eje vertical de los gráficos se representa la distancia en metros a los objetos detectados, mientras que en el eje horizontal se muestran las lecturas en dirección frontal. Se ha trazado una línea roja horizontal a la altura de un metro, la cual indica un umbral de distancia. Si una lectura del sensor LiDAR se encuentra por encima de este umbral, significa que el espacio en esa dirección es transitable. En cambio, si la lectura está por debajo, indica la presencia de un obstáculo demasiado cercano, lo que impide el avance en esa dirección.

En la figura 1 se presentan las lecturas del sensor LiDAR cuando el robot se encuentra frente a una pared. A pesar de esta obstrucción, aún dispone de una ventana de paso, destacada con una marca verde en la parte izquierda de la imagen.

4 EVALUACIÓN DEL COMPORTAMIENTO

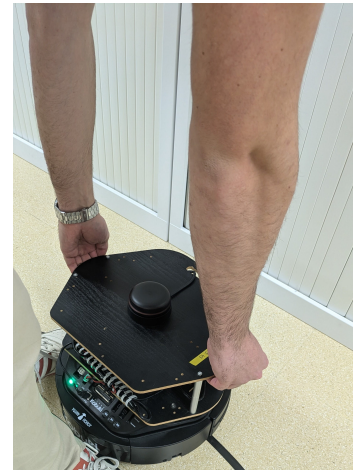
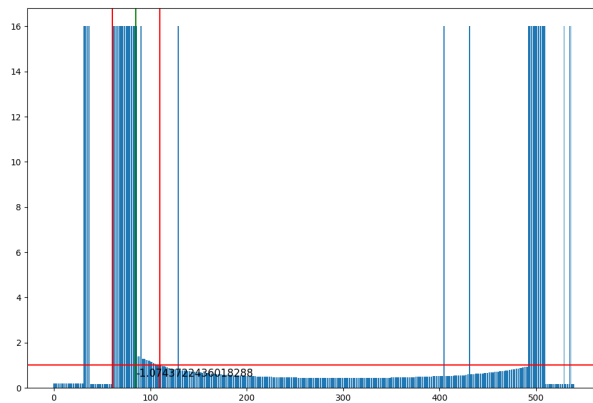


Figura 1: Lecturas de las distancias del sensor LiDAR en dirección frontal, con el robot ubicado frente a una pared.

Por otro lado, en la figura 2 se presentan las lecturas del sensor LiDAR cuando el robot se encuentra en un entorno despejado, sin obstáculos cercanos. En este caso, dispone de múltiples opciones de avance. Como se indica con la marca verde en el gráfico, el robot elige continuar en línea recta.

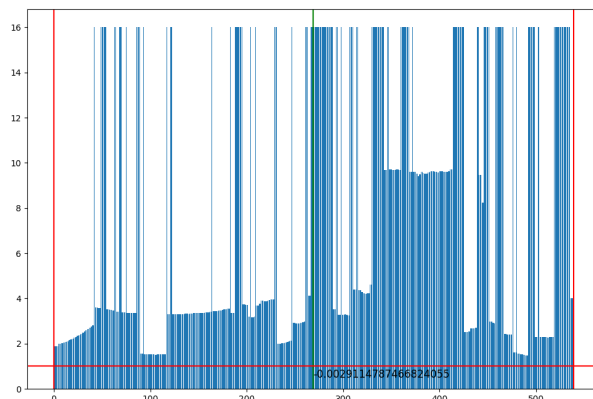


Figura 2: Lecturas del sensor LiDAR en dirección frontal con el robot en un espacio abierto.

Para poder evaluar de manera más sencilla el comportamiento del robot, se ha grabado un video que se pueda consultar utilizando este enlace.

En general, el robot funciona correctamente, cumpliendo su objetivo principal de evitar colisiones con los obstáculos del entorno. Además, no se choca con ningún objeto, incluso cuando se encuentra en un pasillo o cuando algo se coloca justo frente a él. A pesar de todo, a veces el robot da giros bruscos aunque no sea del todo necesario, cosa que se pueda mejorar tal y como se va a mencionar en el apartado 5.

5. Mejoras posibles

Hay varias mejoras posibles que se pueden realizar. Una de estas es no elegir siempre el centro de la ventana encontrada como punto al que avanzar. A veces la nueva ventana encontrada puede estar bastante alejada de la ventana anterior, provocando que el robot gire de manera brusca. Esto se podría solucionar guardando una memoria de la anterior ventana, y eligiendo en la nueva ventana el valor más cercano a la ventana anterior, para luego ir cambiando el punto objetivo poco a poco hacia el centro de la ventana. De esta manera el movimiento sería mucho más fluido y los giros bruscos se realizarían en cuando fuera realmente necesario.

Otra posible mejora consiste en extender el comportamiento de búsqueda de la mejor ventana también al movimiento en reversa, en lugar de retroceder directamente sin evaluar el entorno. Actualmente, cuando el robot se encuentra sin una ruta despejada hacia adelante, simplemente se desplaza hacia atrás sin considerar si hay obstáculos en esa dirección. Implementar la misma estrategia de detección de ventanas óptimas al moverse en reversa permitiría al robot tomar decisiones más seguras y evitar posibles colisiones con paredes u otros objetos detrás de él. Esto le permitiría no solo retroceder, sino también ajustar su trayectoria para encontrar un camino más despejado, mejorando así su capacidad de navegación en entornos complejos.